Farm Profit Simulator

Farm Profit Simulator *(student proposal)*

> **Assessment:** ◆ Fit with adjustments · **Category:** Simulator · **Assessed:** 2026-07-05 · **Status:** awaiting instructor approval

Proposal (as submitted): A farming simulator on a 1-acre farm raising various animals to determine which is most profitable. The farm is maintained with plantations, trees and animal feed; scenarios define starting numbers and maximum numbers of animals.

Adjustments required for approval:

- Animals and plants must be **simulated agents with per-tick behavior** (feeding, growth, breeding, movement in pens, resource consumption) on a rendered 2D farm — not closed-form profit formulas. Without this, the real-time C++ loop is decorative and the project collapses into a calculator.
- The profit comparison must come from **seeded, reproducible batch runs with statistical reports** (the category's Headless Batch Mode requirement) — "which animal profits most" is an experiment across N runs, not one anecdotal run.

References:

- Farming Simulator (GIANTS Software, official)
- Equilinox — ecosystem sandbox (official)
- NetLogo — agent-based modeling platform (official)

Core Tech Requirements

Legend: ✔ Applies directly | ◆ Applies with domain adaptation | ● Optional | ✘ Not needed | » Second trimester

Core Tech Requirement	Farm Profit Simulator
External Database (Firebase)	✔
Authentication »	●
Analytics »	✔
C++ Performant Service	✔
Real-time C++ Simulation Loop	✔
Application Communication	✔
Continuous Integration	✔
Data-Driven Architecture	✔
Front-end + Local Backend	✔
Custom Tools Development	✔
2D Graphical Output	✔
3D (strong students only)	✘
Networking System	✘
Headless Batch Mode — run N simulations without rendering + statistical reports (new)	✔

How each requirement applies:

> *Platform fit:* **Web browser on mobile** — configure scenarios and watch time-accelerated runs on the phone (WASM build); heavy batch experiments run on the PC development build (Native PC is the development vehicle, not a shipping target).

Front end: 2D farm view (pens, plots, trees, animals), run controls, live profit/population charts. **Languages:** C++, GLSL.

Back end: Agent simulation engine (animal/plant lifecycles, feed economy), scenario system, batch runner & statistics, Firebase scenario/result library. **Languages:** C++.
Backend scope: Local now, server-ready (T2) — the batch engine could run server-side with the UI as a thin viewer.

- *Simulation loop:* fixed-timestep agent simulation — animals feed, grow, breed and die; plants and trees grow and yield; the feed economy updates every tick with time acceleration.
- *Data-driven:* species, plants, feed types, prices, starting/max counts and scenario definitions all as data — the profit experiment IS a data file.
- *Custom tool:* farm & scenario editor — lay out the acre (pens, plots, trees) and define species parameters and economics.
- *External DB:* shared scenario and result library. *Auth »:* optional accounts. *Analytics »:* aggregate run statistics.
- *2D:* farm rendering with animated agents plus custom-rendered profit/population charts. *3D / Networking:* not needed.
- *Headless batch mode:* the core question — "which animal profits most on 1 acre?" — is answered by seeded batch runs across species/configurations with confidence-interval reports.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Simulator pool in `milestone-goals-pool-per-category.md` (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**
  Tech & dependencies
  - **Tech:** SDL2 window/input with a fixed-timestep loop and time acceleration (1×–1000×) driving all farm agent updates; C++17/20, CMake, Git.
  - **Prerequisites:** None — foundation of this project.
  - **Feeds into:** Every other goal; M1 G5 behaviors tick inside this loop and M2 G7 reuses it headless.
- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**
  Tech & dependencies
  - **Tech:** Species, plants, feed types, market prices and scenario definitions fully described in JSON (nlohmann/json) with hot-reload for rapid experiment tuning.
  - **Prerequisites:** G1 (loop & update hooks).
  - **Feeds into:** M1 G5 (species parameters), M1 G6 (scenarios), M2 G2 (editor edits this data), M2 G7/G16 (batch configs).
- **G3 ★ Debugging, Profiling, Stress Test & CI**
  Tech & dependencies
  - **Tech:** ImGui overlays (agent counts, tick ms, economy totals), Tracy profiling, Catch2 stress tests with thousands of agents at max acceleration, GitHub Actions Windows+Linux matrix.
  - **Prerequisites:** G1 (loop to instrument).
  - **Feeds into:** M2 G3 (CI hardening); the determinism checks that M2 G7 batch comparisons depend on.
- **G4 2D Visualization of the Simulated World** — render the acre: pens, plots, trees, animated animals; the visual proof the sim is real.

Tech & dependencies

- **Tech:** SDL2/OpenGL farm renderer — pens, plots, trees, animated animal sprites — with camera pan/zoom and FreeType labels/tooltips.
- **Prerequisites:** G1 (loop & window).
- **Feeds into:** The live demo view; M2 G2 embeds it as the editor's live preview.

- **G5 Agent / Entity Behavior System** — animal lifecycles (feed, growth, breeding, mortality) and plant growth as per-tick agent behavior.
  Tech & dependencies
  - **Tech:** Per-tick state machines for animals (feed, grow, breed, die) and growth cycles for plants/trees, seeded RNG for reproducibility, GLM math.
  - **Prerequisites:** G1 (fixed timestep), G2 (species data).
  - **Feeds into:** All profit outcomes; M2 G7 runs exactly these behaviors headless at scale.
- **G6 Scenario & Parameter System** — species parameters, starting/max numbers, feed costs and market prices as editable scenarios.
  Tech & dependencies
  - **Tech:** Scenario schema (starting/max animal counts, feed costs, prices, plot allocation) with validation and quick-swap presets for A/B experiments.
  - **Prerequisites:** G2 (data model).
  - **Feeds into:** M2 G2 (editor authors scenarios), M2 G7 (batch inputs), M2 G16 (campaign sweeps).

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**
  Tech & dependencies
  - **Tech:** Tracy-guided optimization of agent updates (pooling, cache-friendly layouts), ASan on MSVC; WASM build validated on a mobile browser (reference phone Pixel 8a/9a), plus PC validation at maximum time acceleration with thousands of agents.
  - **Prerequisites:** All M1 goals (hardens the M1 sim codebase).
  - **Feeds into:** M2 G7 (batch throughput — more runs per minute means better statistics).

- **G2 ★ Content Editor — Core** — the farm & scenario editor: acre layout, species parameters, economics tables, with JSON round-trip and undo/redo.

  Tech & dependencies
  - **Tech:** Farm & scenario editor — lay out pens/plots/trees on the acre, edit species and economics tables — with JSON round-trip and an undo/redo command stack.
  - **Libraries/Software:** Dear ImGui, nlohmann/json, project's SDL2/OpenGL farm view (live preview)
  - **Prerequisites:** M1 G2 (data model), M1 G6 (scenario schema), M1 G4 (farm view).
  - **Feeds into:** Every configured experiment; M2 G16 campaigns are authored here.
- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

  Tech & dependencies
  - **Tech:** CMake presets, clang-format + clang-tidy gates, CPack packaging, GitHub Actions matrix including a headless batch smoke test.
  - **Prerequisites:** M1 G3 (CI and test foundation).
  - **Feeds into:** Release/showcase builds; the CI batch test guards M2 G7 determinism.
- **G7 Headless Batch Mode & Statistical Reports** — the profit question answered properly: N seeded runs per configuration, means and confidence intervals.

  Tech & dependencies
  - **Tech:** Render-free simulation runs (N seeds × configurations), aggregation of profit/population series into means and confidence intervals, CLI entry point.
  - **Prerequisites:** M1 G5 (agents), M1 G6 (scenarios), M2 G1 (throughput).
  - **Feeds into:** M2 G15 (reports) and M2 G16 (campaigns) — this goal produces the project's answer.
- **G15 Report Generation & Export Tools** — species-comparison profit reports (charts + tables) exportable for the showcase.

Tech & dependencies

- **Tech:** Custom-rendered comparison charts (profit per species over time, distributions) and tables, exported as PNG/CSV/HTML report bundles.
- **Prerequisites:** M2 G7 (statistics to report).
- **Feeds into:** The showcase deliverable — "goat vs. sheep vs. chickens on 1 acre" as a defensible report.

- **G16 Multi-Scenario Campaign Runner** — sweep species mixes and starting/max counts across scenario campaigns to find the optimum.
  Tech & dependencies
  - **Tech:** Campaign definitions sweeping species mixes and count ranges, a job queue over the batch runner with progress reporting and resume.
  - **Prerequisites:** M2 G7 (batch runner), M1 G6 (scenario schema).
  - **Feeds into:** The optimum-farm result presented at the demo.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 1 Art — a stylized, watchable farm world (animated animals, crops, weather touches) is exactly the Simulator case where 1 artist is justified.

> **Track goals** — suggested Design/Art picks and TEAM notes for this project; deliverables & quality ladders live in the Design, Art and TEAM pool documents.
>
> - **Design M1:**
>   - D1 ★ Features DD — the farm model, who reads the profit insights and for what decisions, fidelity-vs-charm trade-offs
>   - D2 Prototype — configure a season → run → read profit results, clickable end to end
> - **Design M2:**
>   - D1 ★ Features Design Iteration & Showcase Readiness
>   - D6 Polished experiment — one full season + species-comparison batch, end to end for the showcase
> - **UI/UX M1:**
>   - U1 ★ UI Standards & Wireframe Baseline — run-view and report flows, mobile-web readability standards
> - **UI/UX M2:**
>   - U2 ★ UX Validation & Final Polished UI — UX validation with users interpreting season reports; final polished UI on mobile web
>   - U1 ★ Functional UI & Usability — Simulator screens: scenario select · run view with pause/speed · season/batch report · settings · help; Resume/Quit apply to run sessions
> - **Art M1:**
>   - A1 ★ (mockups: farm live view + season report screen)
>   - A2 Agent sprites — animal/crop/tree sprites with readable growth stages
> - **Art M2:**
>   - A1 ★ Critical asset set in the showcase build
>   - A3 Living farm — ambient motion (grazing, swaying crops, weather) that makes the sim watchable

- **TEAM M1:**
  - TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present and working in the prototype, suitable and correctly triggered for this project.
  - TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add or replace audio without recompiling) and reliable multi-channel playback.
- **TEAM M2:**
  - TA1 ★ Overall Audio Experience — Simulator floor: BGM optional (ambient bed); ≥5 distinct event cues (run start/stop, harvest, alert, completion), non-intrusive and legible
  - TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel; advanced where the project calls for it).
  - TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase, prototype demo, team post-mortem (wk14)

Generated by the Project Proposal Assessor skill · grounding: live folder · references verified 2026-07-05 · track goals & unified workbooks retrofitted 2026-07-12.